

7.0 Detailed Jobs Crawling – JPERL Mode

1. What is JPERL?

JPERL Mode is a backend crawling mechanism used to extract jobs from client career pages when:

- Jobs are not easily accessible via simple HTML/XPath (SRP Mode)
- Data is rendered through **JavaScript, JSON APIs, or embedded script blocks**
- The page requires **regex, JSON parsing, or special headers/POST body** to fetch job data

In JPERL, you configure **rules and patterns** instead of writing code. The crawler then:

1. Fetches HTML / JSON from the page or API
2. Applies **JSON mappings or regex** to extract:
 - Job ID
 - Job Title
 - Location
 - Job Link
 - Posted Date
 - Job Description
3. Saves the jobs into the internal database

2. When to Use JPERL (Feasibility & Decision)

You choose **JPERL** when **SRP Mode** is **not suitable**, for example:

◆ Use JPERL for:

- **ATS systems** like:
 - Workday, Lever, Greenhouse, Darwinbox, SuccessFactors, Oracle, BambooHR, Ceipal, Keka, etc.
- **JS-heavy / API-driven sites**:
 - Where job data loads via XHR/API calls
- **JSON-based or script-embedded job data**
- **Sites hosted on platforms** like:
 - Cloudflare, Hostinger, Microsoft services, etc.
- **Plain HTML sites** where **SRP** cannot detect stable job blocks

◆ When NOT to use JPERL:

- Page is **Robot-blocked**
 - No job content is accessible
 - Completely static brochure sites with no job listing
-

3. Inputs & Sources – Where You Start

3.1 Get Company IDs

You normally pick companies for JPERL when SRP fails or is not possible.

- From **OMS Company List**
- From **OMS Dashboard**
- From **Tech-Rectification group on MS Teams** (Mapping team shares new client IDs)

🔗 OMS JPERL Dashboard

<https://naukrics.infoedge.com:8083/#/techops/perl>

3.2 Robot Check (Compulsory Before Config)

Some sites are blocked via `robots.txt`. Those **cannot be crawled** by any mode (SRP/JPERL/UC).

Use internal Robot Checker:

🔗 <http://192.168.2.123:8015/checkRobot?url=>

- Paste **career page URL**
- If response or log shows:
- Response Body returned by `robots.txt` checker: 2

→ **Site is robot-protected** → mark as **ROBOT** in OMS → do not configure JPERL.

4. Analysing the Career Page (Tech Analysis Steps)

Before writing any JPERL config:

1. **Open career page in browser**
2. Press **F12** → open **Developer Tools**
3. Go to **Network** tab
4. Refresh the page

Now look for:

- **XHR / Fetch / API calls** returning JSON (e.g., jobs list)
- **HTML** containing `<div>`, `<li>`, `<a>` job cards
- **<script>** tags that contain JSON job arrays
- **Pagination patterns** (`page=1`, `offset=20`, etc.)
- Whether **JD is on same page** or **separate JD URL**

Try to answer:

- Is job data in **JSON**?

- Is job data only in **HTML**?
 - Is there **pagination**? How?
 - Is JD in the same document or a **new URL**?
-

5. Tools You Use with JPERL

5.1 Browser Developer Tools

- Check **network calls**
 - See response data (JSON/HTML)
 - Inspect **headers & request payload**
-

5.2 Regex Tool – regex101

 <https://regex101.com/>

Use this to:

- Write and test regex for:
 - Job blocks
 - JD segments
 - IDs, titles, locations, etc.

Example regex you shared:

```
(?s)"id":([^\,]+).+?location":"([^\"]+).+?designation":"([^\"]+).+?experience":"([^\"]+)[^\"]+"qualification":"([^\"]+).+?description":"(.*?)</p>"}
```

This extracts:

1. ID
 2. Location
 3. Title/Designation
 4. Experience
 5. Qualification
 6. JD
-

5.3 JSON Formatter – jsonformatter

 <https://jsonformatter.org/>

Use to:

- Beautify JSON
- Understand nested structure
- Identify keys like:
 - id

- title
 - location
 - description
-

5.4 HTML Viewer – [htmledit.squarefree](https://htmledit.squarefree.com/)

🔗 <https://htmledit.squarefree.com/>

Use this to:

- Paste JD HTML
 - Visualize tags
 - See `<p>`, `<ul>`, `<li>`, etc.
 - Clean up complex/nested HTML
 - Decide which tag to capture via regex for JD
-

5.5 Sublime Text

Use Sublime to:

- Paste full HTML / JS from pages
 - Remove comments, `<script>`, noise
 - Format minified code
 - Extract the section containing job list or JD
 - Make regex easier by working on a clean sample
-

5.6 curlconverter + Postman

- 🔗 <https://curlconverter.com/> – converts browser network requests to cURL/Python/Postman
- **Postman** – test APIs with headers + POST body

Use when:

- Site needs **headers**, **cookies**, **tokens**, or **POST data** to return jobs.
-

6. JPERL Configuration Concepts

This is the **brain** of JPERL crawling.

You configure key–value pairs like:

- Where to find jobs
- How to parse them (JSON or regex)
- How to extract JD

- How to paginate
- Whether to use proxy
- How to format dates

Below is a dictionary of the main parameters.

6.1 LOCJSON & LOCJSONSEQ – JSON-Based Job List

When to use:

Job list is in JSON, e.g.:

```
{
  "jobs": [
    {
      "id": "123",
      "title": "Software Engineer",
      "location": "Bangalore",
      "description": "<p>JD here</p>"
    }
  ]
}
```

Parameters:

- LOCJSON – path in JSON or how data is accessed (internal config)
- LOCJSONSEQ – list of fields to pick, in order

Example:

LOCJSONSEQ = JOBID, JOBTITLE, JOBLINK, LOCATION, JOBDESC

JPERL expects your JSON extraction to map each key to these fields.

6.2 LOCRGX & LOCRGXSEQ – Regex-Based Job List

When to use:

Job list is in HTML, not JSON. For example:

```
<div class="card card-job">
  <a href="/careers/123">Senior Engineer</a>
  <span class="location">Bangalore</span>
</div>
```

Regex Example (from your Riyo Advertising config):

```
(?s)<div class=\"card card-
job\">[^\"]+\"[^\"]+\"[^\"]+\"((\"[^\"]+\"))\">[^>]+>[^>]+>([^\<]+)<[^\>]+>[^>]+>[^\>]+>([^\<]+)<
```

This might extract (as capturing groups):

1. JOBID or internal key
2. JOBLINK or related value

3. JOBTITLE
4. LOCATION

Then you define:

```
LOCRGXSEQ = JOBID, JOBLINK, JOBTITLE, LOCATION
```

So the crawler knows which group is which field.

6.3 JDJSON1 & JDJSONSEQ1 – JD from JSON

When to use:

- JD is in a separate JSON object/endpoint
- Job list is in LOCJSON, but JD is only available via JDJSON

Example:

```
JDJSON1 - JSON that contains "description": "..."  
JDJSONSEQ1 = JOBDESC
```

6.4 JDRGX1, JDRGXSEQ1 – JD from HTML via Regex

Used when JD is stored in HTML sections.

Example from your config:

```
"JDRGX1": "(?s)<h1>Job Title:(.*?)contact-info\\>",  
"JDRGXSEQ1": "JOBDESC"
```

This regex will capture all content between `<h1>Job Title:` and `contact-info">` as JD.

You can define multiple JD regex patterns:

- JDRGX1 / JDRGXSEQ1
- JDRGX2 / JDRGXSEQ2

...to support different page layouts.

6.5 ESCAPE_VARJOBLINK

Sometimes JSON job links look like:

```
"https:\\\\company.com\\careers\\job\\123"
```

With `ESCAPE_VARJOBLINK = No`, JPERL converts this to:

```
https://company.com/careers/job/123
```

So that JOBLINK becomes a valid URL.

6.6 Pagination – !0o!CURPG!0o! and !0o!STARTJOBNO!0o!

Page-based pagination:

URLs like:

```
?page=0  
?page=1  
?page=2
```

Use:

```
!0o!CURPG!0o!
```

Example:

```
URL = https://site.com/careers?page=!0o!CURPG!0o!
```

Offset-based pagination:

URLs like:

```
offset=0  
offset=10  
offset=20
```

Use:

```
!0o!STARTJOBNO!0o!
```

Also set:

```
"MAXPAGESPARSE": 10
```

So JPERL knows how many offset jumps to make.

6.7 Multiple URL Inputs – !0o!URLVARS!0o! and URL_VARS

Use Case:

Different locations or departments require different URLs.

Example:

```
URL_VARS = bangalore!@@!noida!@@!hyderabad  
URL = https://site.com/jobs?location=!0o!URLVARS!0o!
```

Crawler will run the same logic for each location.

6.8 PROXY = "Yes"

Use when:

- Site is blocked on server
- Logs show 403/503 or no response
- Client is accessible only via proxy

Be careful – proxy usage consumes quota.

6.9 JDVARS – Classifiers

From your example:

```
"JDVARS": {  
  "FAREA": { "callable": "Classifier_FAREA", "type": "SUB" },  
  "INDUSTRY": { "callable": "Classifier_INDUSTRY", "type": "SUB" },  
  "ROLE": { "callable": "Classifier_ROLE", "type": "SUB" }  
}
```

These are internal classifiers that run on the JD text to classify:

- Functional Area
 - Industry
 - Role
-

6.10 Other Common Fields

- CanProceed: flag to allow execution
 - DELETE_UNLISTED: "y" → delete jobs not found this run
 - MOVE_TO_JD: 1 → indicates JD comes from separate page
 - JOBLINK: usually uses {{VARJOBLINK}}
 - SQLDATE: date format, e.g. %Y-%m-%d
 - USER_AGENT: e.g. "Firefox/5.0"
 - PROXY: "Yes" for proxy usage
 - POSTQUERY: SQL to update comp name/ID, etc., after crawl completes
-

7. Predefined ATS Rules (Very Important)

For some ATS platforms, **JPERL rules are already predefined** in backend.
You usually **only change**:

- URL (main URL)
- URL_VARS (company name / domain code)

No need to write regex or JSON again.

Some of these include:

- Greenhouse → <https://boards.greenhouse.io/<company>>
- Lever → <https://jobs.lever.co/<company>>
- BambooHR → <company>.bamboohr.com/careers
- Ceipal / Cieple
- Keka HR
- and similar ATS

That's why for those ATS types:

You mostly just plug in URL and URL_VARS, rest is handled by predefined JPERL templates.

8. End-to-End Example – Riyo Advertising (4014506_SRP)

Here's the **full example config you shared**, annotated conceptually.

```
{
  "4014506_SRP": {
    "CanProceed": 1,
    "Cookie": "YES",
    "CurrentJobNum": 1,
    "CurrentPageNum": 0,
    "DELETE_UNLISTED": "Y",
    "ISCRAWL_FULL": 0,
    "IS_SITE_ID_DERIVED": 0,

    "JDRGX1": "(?s)<h1>Job Title:(.*?)contact-info\\>",
    "JDRGXSEQ1": "JOBDESC",

    "JDVARS": {
      "FAREA": { "callable": "Classifier_FAREA", "type": "SUB" },
      "INDUSTRY": { "callable": "Classifier_INDUSTRY", "type": "SUB" },
      "ROLE": { "callable": "Classifier_ROLE", "type": "SUB" }
    },

    "JOBLINK": "{{VARJOBLINK}}",
    "JOBSCOUNT": 0,

    "LOCRGX": "(?s)<div class=\\\\"card card-
job\\\\">[^\\"]+\\\\"[^\\"]+\\\\"[^\\"]+\\\\"(([^\\"]+))\\\\">[^>]+>[^>]+>([<>]+)<[^>
+>[^>]+>[^>]+>([<>]+)<",
    "LOCRGXSEQ": "JOBID, JOBLINK, JOBTITLE, LOCATION",

    "MAXJBSPPAGE": 10,
    "MAXPAGESPARSE": "1",

    "MOVE_TO_JD": 1,
    "POSTQUERY": "update WEB_JOBS set COMPNAME ='Riyo Advertising',compid
='4014506', jobConsultant = 'n' where SITE = '4014506_SRP'",

    "PROXY": "Yes",
    "SAVE_JD_COOKIE": "Yes",

    "SQLDATE": "%Y-%m-%d",
    "StartPageNum": 0,

    "TERMRGX": "",
```

```

    "URL": "https://riyoadvertising.com/CurrentOpening.shtml",
    "USER_AGENT": "Firefox/5.0"
  }
}

```

What this configuration does:

- Hits `https://riyoadvertising.com/CurrentOpening.shtml`
- Uses `LOCGRX` to extract:
 - `JOBID`
 - `JOBLINK`
 - `JOBTITLE`
 - `LOCATION`
- For each job, opens `JOBLINK` (because `MOVE_TO_JD = 1`)
- Uses `JDRGX1` to extract `JD` between `Job Title:` and `contact-info">`
- Runs `JDVARS` classifiers on `JD`
- Marks jobs with `compname = Riyo Advertising & compid = 4014506`
- Uses `PROXY` because site is blocked on server

9. Running JPERL & Checking Logs

9.1 After Creating Config – Hit RUN

Then check NIC logs:

`http://192.168.2.123/nicrawledlogs/crawled/YYYY-MM-DD/siteID_UC/siteID_SRP.html`

Replace:

- `YYYY-MM-DD` with today's date
- `siteID_SRP` with e.g., `4014506_SRP`

In logs, check:

- Job count found
- Any regex errors
- Any robot message
- Any 403/503
- JSON parse errors
- Whether `JD` extraction worked

10. QC & Validation Flow (After JPERL Crawl)

Once backend crawl is done (usually next morning):

1. Open **QC Interface**
2. Check:
 - **New/Modified Jobs Pool**

- **Published Jobs Pool**
- **Hold Jobs Pool**
- 3. Validate:
 - Title
 - Location
 - JD accuracy
 - Links open correctly
 - No duplicate or junk jobs
- 4. Move invalid jobs → **Hold**
- 5. Publish valid jobs
- 6. Mark job crawling as **Done** in dashboard

JPERL Issue Escalation & Tracking Process

When Jobs Do Not Appear After JPERL Crawl

If the crawler does not fetch jobs the next morning (missing jobs, zero jobs, partial JD, broken regex, or JSON not loading), follow the mandatory escalation workflow below.

1. Log Issue in the Official JPERL Issues Sheet

All JPERL extraction failures must be logged in the central tracking sheet:

★ JPERL Issues Sheet (SharePoint)

🔗 https://infoedge-my.sharepoint.com/:x/r/personal/samyak_srivastava_infoedge_com/_layouts/15/Doc.aspx?sourcedoc=%7B81FFCF4D-30AA-4674-8DE8-407E7879EEFE%7D&file=JPerl%20Issues.xlsx&action=default&mobileredirect=true

2. Fill the Issue Details in the Following Format

Field	Description
Date	Date when the issue is noticed
siteID	The JPERL site ID (e.g., 4014506_SRP)
Issue	Describe the exact problem (e.g., <i>0 jobs fetched, JD missing, regex mismatch, 404 error, JSON blank, Request blocked</i>)
Interface	Mention where the issue is visible (<i>QC Interface, Logs, Career Page</i>)
Issue Raised By	Your name / TechOps executive reporting
Expected Jobcount	Job count visible on the client's career site
Issue Cause	Write the suspected cause if known (<i>regex broken, structure changed, need proxy, API blocked, robot issue, etc.</i>)

This helps backend engineers quickly diagnose and fix the problem.

3. Mention Likely Cause (If Known)

While logging the issue, include one of the common failure causes:

- **JSON structure changed** (keys modified / nested JSON)
- **Regex mismatch** due to UI redesign
- **JD API endpoint changed or removed**
- **CORS blocked / server-side restrictions**
- **403 / 503 errors in logs**
- **Robots.txt disallows crawling**
- **Proxy required** (site blocked on server)
- **POST request needs headers / dynamic parameters**
- **Career page temporarily down**
- **Pagination changed** (offset/page/limit changed)

Even if unsure, providing hints helps the backend team troubleshoot faster.

4. Proxy Usage Rule (IMPORTANT)

- ◆ Some client sites block requests originating from the server.
- ◆ In such cases, **PROXY = Yes** must be added in JPERL configuration.
- ◆ However, proxy consumes paid usage, so:

Proxy should be enabled ONLY after approval from Backend Tech Team.

Never use proxy without explicit technical approval.

This prevents unnecessary proxy consumption and ensures controlled usage.

5. Backend Team Responsibilities After Issue is Logged

Once the issue is added to the sheet:

- Tech Team checks logs & crawler behaviour
 - Reviews **JSON / regex / URL / headers**
 - Updates configuration if required
 - Enables **proxy** only if necessary
 - Reruns JPERL crawler
 - Adds remarks in the Issues Sheet
 - Hands back configuration to TechOps for validation
-

6. If Still Not Resolved → Perform Manual Crawling

If the backend team confirms the site cannot be crawled due to:

- Strong robot protection
- Encrypted JS data
- Cloudflare / Hostinger blocking
- Captcha walls
- Closed API
- ATS restrictions
- Dynamic rendering not detectable by crawler

Then the crawling must be done in Manual Mode.

Add remark in OMS:

"JPERL not feasible – switched to Manual Crawling."